SMART ENERGY INTELLIGENCE PLATFORM

Technical Documentation

SIC 8 — Big Data Engineering Graduation Project

Github repo: https://github.com/adhamtak/smart-energy-platform

Team

1.Adham Tarek Abdel Khaleq
2.Mariam Mahmoud Talaat
3.Shahd Mohamed Ibrahim
4.Mennatullah Mahmoud

Smart Energy Intelligence Platform

SIC 8 — Big Data Engineering Graduation Project

1. Executive Summary

The Smart Energy Intelligence Platform simulates an electricity distribution environment in which smart meters produce interval-based readings. It provides a reproducible data foundation for historical analytics, continuous ingestion, anomaly detection, and operational monitoring.

This document focuses on synthetic reference data and readings, historical and continuous generation, Kafka publishing, raw-data delivery to MinIO, validation, and handover. Batch transformations, streaming transformations, analytical models, and dashboards are downstream responsibilities and should be labelled implemented only after their behavior has been tested.

2. Introduction

The Smart Energy Intelligence Platform is a data engineering project that simulates how an electricity distribution company can collect, process, and analyze large volumes of smart-meter data.

The platform simulates an electricity distribution environment containing 100,000 smart meters that generate electricity readings at 15-minute intervals. It combines historical data generation with continuous streaming ingestion to demonstrate how large-scale energy data can be collected and prepared for analysis.

The synthetic dataset is designed using real-world electricity distribution concepts and publicly available technical references, including smart-meter specifications published by the Egyptian Electricity Holding Company (EEHC). These references guide the selection of realistic data attributes and help the simulation reflect electricity distribution scenarios.

The project's goal is to build a reproducible data pipeline that supports historical analysis, continuous data ingestion, anomaly detection, and operational monitoring through a dashboard.

3. Problem Statement

Electricity distribution networks produce large volumes of smart-meter readings that can be difficult to collect, organize, and analyze efficiently.

Handling this data introduces several challenges, including:

- Managing large volumes of readings from multiple smart meters.
- Supporting both historical analysis and continuous data ingestion.
- Organizing raw data into a structure suitable for further processing.
- Identifying unusual consumption patterns, voltage violations, and potential transformer overloads.
- Preparing reliable datasets for operational analytics and dashboard visualization.

The Smart Energy Intelligence Platform addresses these challenges by simulating a scalable data pipeline that collects historical and streaming readings, stores the incoming data, and provides a foundation for downstream processing and analytical insights.

4. System Architecture

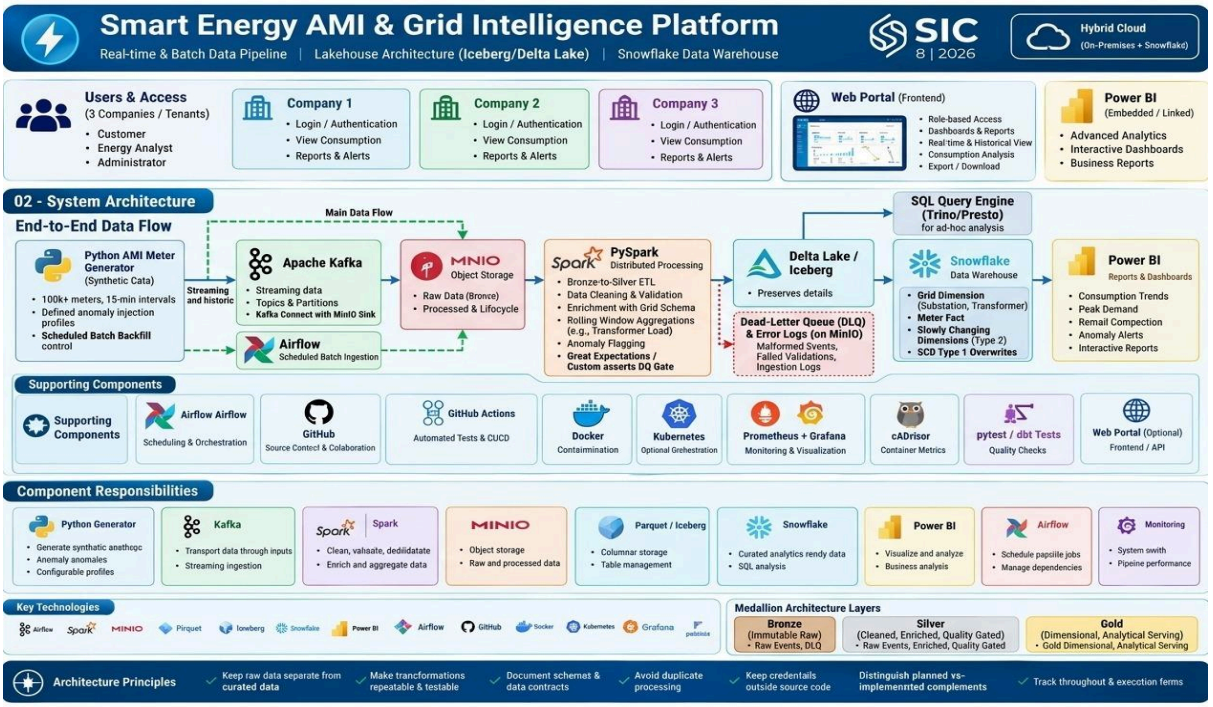
4.1 Architecture Overview

The platform uses Python for synthetic data generation, Apache Kafka for streaming data transmission, and MinIO for S3-compatible object storage. The collected data is intended to be processed through batch and streaming workflows before being used by the analytics and dashboard layers.

4.2 Data Flow

1. **Synthetic Data Generation — Python: Generates historical smart-meter readings and continuously produces new simulated readings.**
2. **Streaming Ingestion — Apache Kafka: Receives streaming readings from the generator and makes them available to downstream consumers.**
3. **Kafka Consumer — Python: Reads incoming messages and writes them to MinIO in Parquet format.**
4. **Object Storage — MinIO: Stores raw data in the Bronze layer and provides the foundation for organizing processed data into Bronze, Silver, and Gold layers.**
5. **Batch Processing — PySpark: Intended to process historical datasets, apply transformations, and produce analytics-ready outputs.**
6. **Streaming Processing — Spark Structured Streaming: Intended to process incoming readings incrementally and support timely anomaly detection.**
7. **Analytics and Dashboard — Power BI: Uses processed datasets to visualize consumption trends, transformer loading, and potential operational anomalies.**

4.3 Architecture Diagram



5. Simulated Data Model

The data model links each meter to a transformer, each transformer to a substation, and each substation to a region. This hierarchy supports aggregation by operating area and helps validate referential integrity.

Entity	*Purpose*	*Previously validated full-scale run*
Region	***Operating region / distribution area***	***9***
Substation	***Groups transformers within a region***	***25***
Transformer	***Distribution transformer***	***547***
Meter	***Customer smart meter assigned to a transformer***	***100,000***
Reading	***Time-stamped electrical measurements***	***9,600,000 for one day***

5.1 Reading Cadence and Volume

Property	Value / interpretation
Historical interval	15 minutes
Intervals per meter per day	96
Meter population	100,000
Expected full-day readings	100,000 × 96 = 9,600,000
Energy calculation	energy_kwh is interval energy, approximately active_power_kw × 0.25
Timestamp convention	UTC timestamps; display in Africa/Cairo time in dashboards where applicable

5.2 Main Reading Fields

Field	Description
reading_id	Unique reading-record identifier
event_timestamp	Timestamp of the reading interval
meter_id	Source meter identifier
transformer_id	Associated transformer identifier
voltage_v	Voltage measurement in volts
active_power_kw	Active power in kilowatts
reactive_power_kvar	Reactive power in kVAr
energy_kwh	Energy for the interval, not a cumulative counter
power_factor	Power factor where available; may be nullable
schema_version	Integer schema version

6. Generation and Data Delivery

6.1 Historical Data Generation

The historical generator creates reference dimensions and interval readings for the simulated distribution network. Output is written locally so it can be validated before being uploaded or consumed by downstream jobs.

Previously validated full-scale command:

```
python .\generator\src\historical_generator.py --meters 100000 --days 1 --start-date 2026-07-02
--output-dir .\data\full-scale-100k-1day --batch-meters 1000
```

Argument	Meaning
--meters 100000	Total meter population
--days 1	Number of historical days to generate
--start-date 2026-07-02	Start date for the generated historical day
--output-dir ...	Destination folder for generated output

--batch-meters 1000	***Processes meters in chunks; it does not limit the total to 1,000 meters***

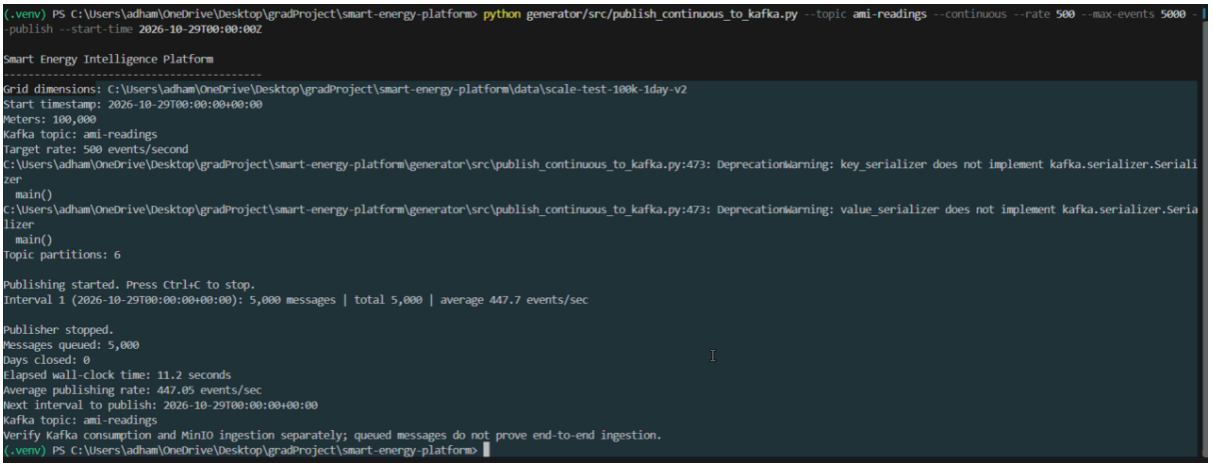
Before rerunning into an existing output folder, confirm whether the script overwrites or reuses files. Use a fresh output directory if behavior is uncertain, and never mix demo and full-scale files.



6.2 Continuous Data Generation

The continuous generator simulates newly arriving readings and feeds the streaming path. Keep this path distinct from the historical backfill. The intended scenario is one new reading per meter per 15-minute interval—not replaying a full historical day every 15 minutes.

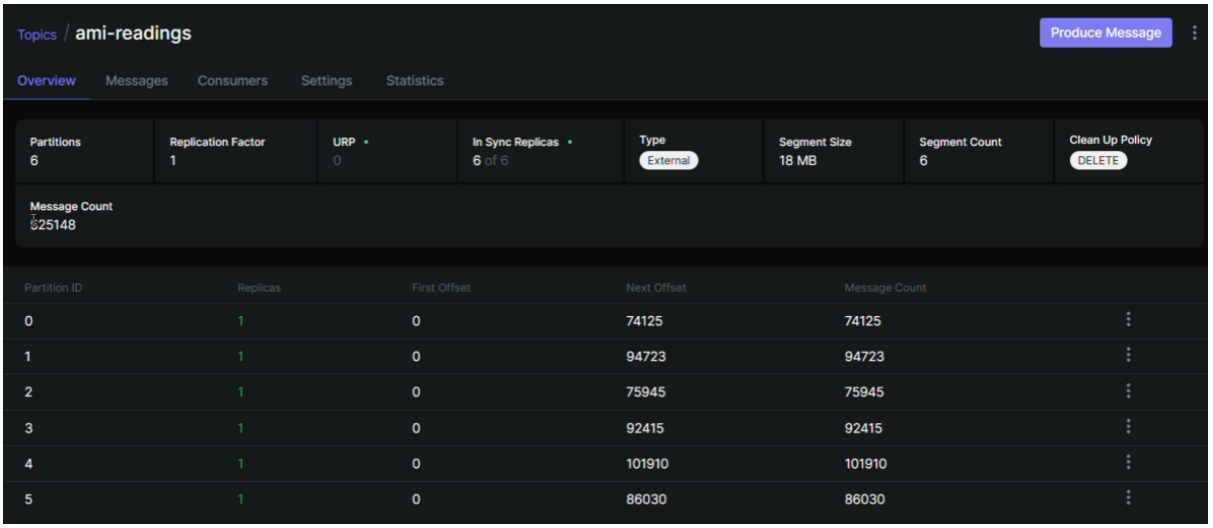
For each demo, record the meter population, Kafka topic, publishing rate, run duration, and whether it is a smoke test or full-scale run.



6.3 Kafka Publishing

The publisher sends generated readings to a configured Kafka topic, separating generation from consumption. Topic names and environment variables must match the consumer configuration. Do not confuse development/test topics with the shared topic intended for the

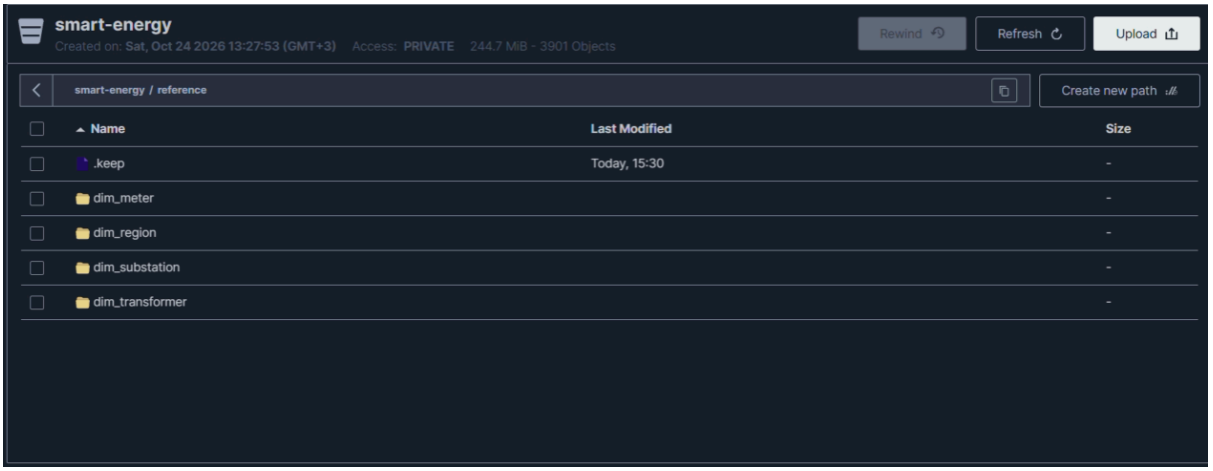
demonstration.

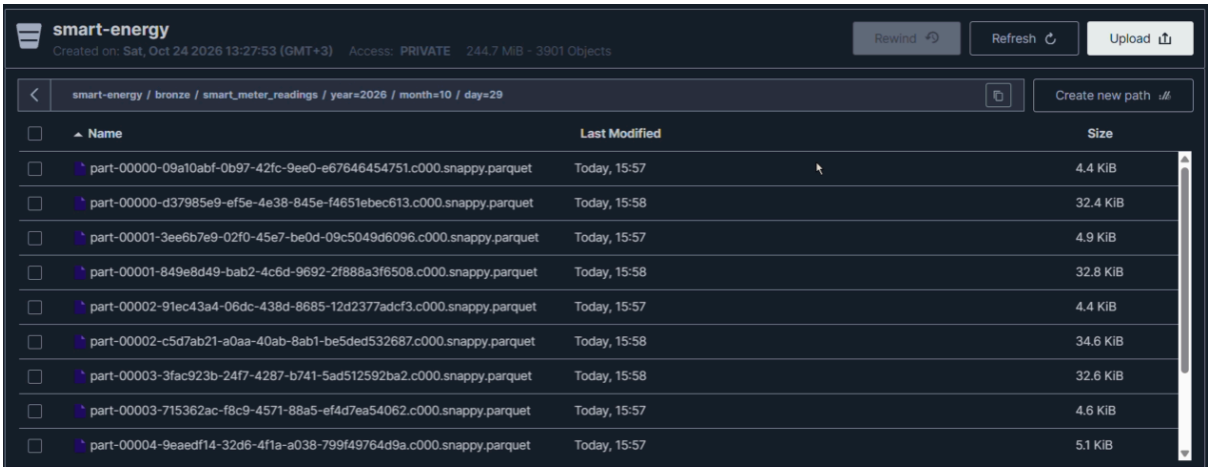


6.4 MinIO Raw Data Delivery

Historical Parquet files and consumed streaming batches are intended to land in MinIO. The upload/consumer configuration and the processing notebook's input path must agree on the bucket and prefix.

A local output folder is not the same as a Bronze dataset. Successful generation proves local file creation; it does not prove that the files were uploaded or that Databricks is reading them.





6.5 Reliability and Testing

Tests should distinguish generator correctness, Kafka publishing, consumer behavior, and successful storage. A successful test at one stage does not prove that later stages are correct.

Test	*What to verify*	*Evidence*
Dimension counts	***9 regions, 25 substations, 547 transformers, 100,000 meters in the validated full-scale setup***	***Generator log or validation query***
Historical volume	***9.6 million readings for one day; 96 readings per meter***	***Generator summary / Parquet validation***
Region coverage	***All nine region codes appear after mapping readings through dimensions***	***Regional count summary***
Relationship integrity	***Meters link to valid transformers; transformers link to valid substations and regions***	***Validation output***
Kafka smoke test	***Records reach the configured topic***	***Publisher log and Kafka UI***
Storage smoke test	***Parquet objects appear in the intended MinIO bucket/prefix***	***Consumer log and MinIO console***

7. Handover to the Processing Team

A handover should identify exactly which run is ready, where the files are stored, and how completeness was checked. The receiving team should not assume that a local output folder is automatically available in Bronze.

Handover item	*Required information*
Run identity	***Run date, historical start date, and output folder or run identifier***
Dataset size	***Meter count, reading count, and time range***
Reference dimensions	***Paths and counts for region, substation, transformer, and meter dimensions***
Storage destination	***Exact MinIO bucket and object prefix after upload***
Schema	***Field names, types, timestamp convention, and schema version***
Validation	***Region coverage, relationship integrity, duplicate IDs, and null checks***
Known limitations	***Partial runs, smoke-test data, or any injected bad data***

Streaming Processing

Purpose

The streaming job reads smart meter readings from Kafka in real time, rejects invalid readings, removes duplicates, stores valid readings in the data lake, and detects three kinds of grid problems, publishing each as an alert.

Data flow

Generator → Kafka topic `ami-readings` → Spark Structured Streaming job → MinIO Bronze folder (valid readings), Kafka topic `ami-readings-dlq` (rejected readings), Kafka topic `anomaly-alerts` (detected problems).

The job is `streaming/stream_alerts_processor.py`. Settings come from `.env`.

What the job does with each message

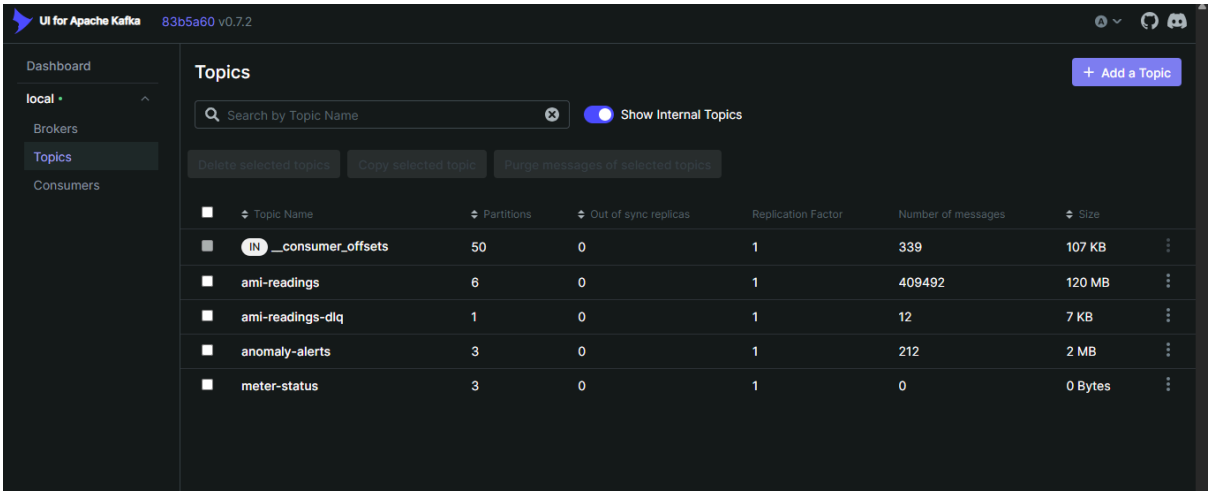
1. Parse the JSON using the shared schema (`shared/schemas.py`).
2. Validate. A message is rejected, with a reason, if it is:
    - not valid JSON or empty (`malformed JSON`)
    - missing a required field (`missing required field`)
    - carrying an unparsable timestamp (`invalid event_timestamp`)
    - carrying `voltage_v` of 0 or less (`voltage_v must be > 0`)
    - carrying negative `active_power_kw` or `energy_kwh` (`negative power or energy`)
3. Rejected messages go to `ami-readings-dlq` with the original payload, the reason, and a UTC timestamp. Unusual but possible values (for example 190 V) are kept, because they may be the anomaly we want to detect.
4. Deduplicate on `reading_id` plus event time, with a 30-minute watermark. Readings more than 30 minutes older than the newest event time seen are dropped.
5. Store valid readings as Parquet in Bronze, partitioned by `year/month/day` taken from the reading's own timestamp.
6. Enrich each reading with `region_id` and `rated_capacity_kva` from the transformer dimension (`reference/dim_transformer/`). This table is read once at startup, so the job must be restarted after the dimension changes.
7. Detect anomalies and publish alerts to `anomaly-alerts`.

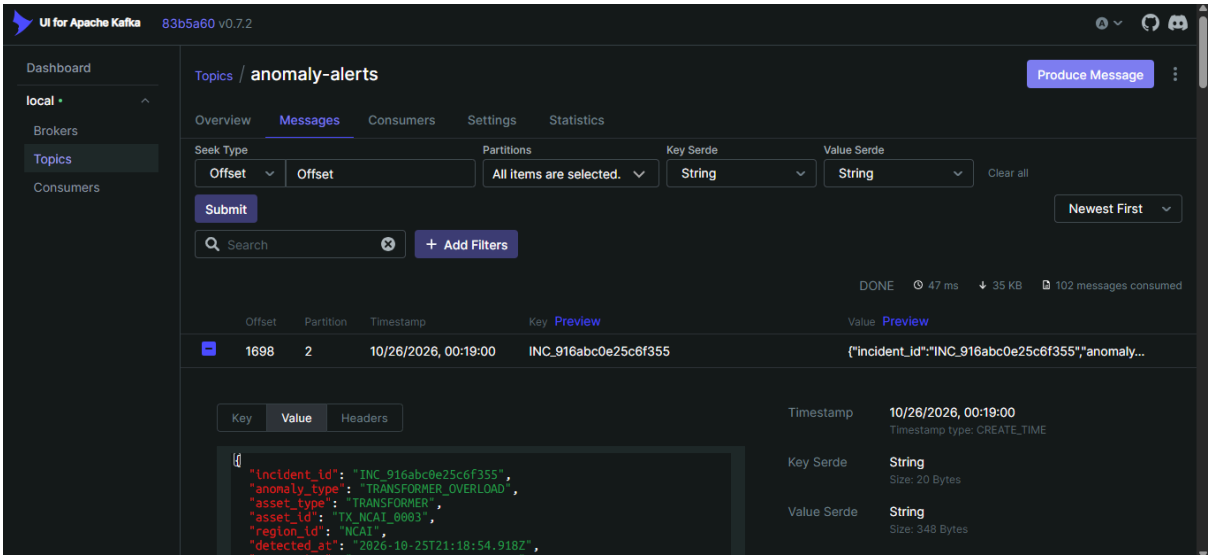
Alert rules

All thresholds are configurable in `.env` and are assumptions to be confirmed.

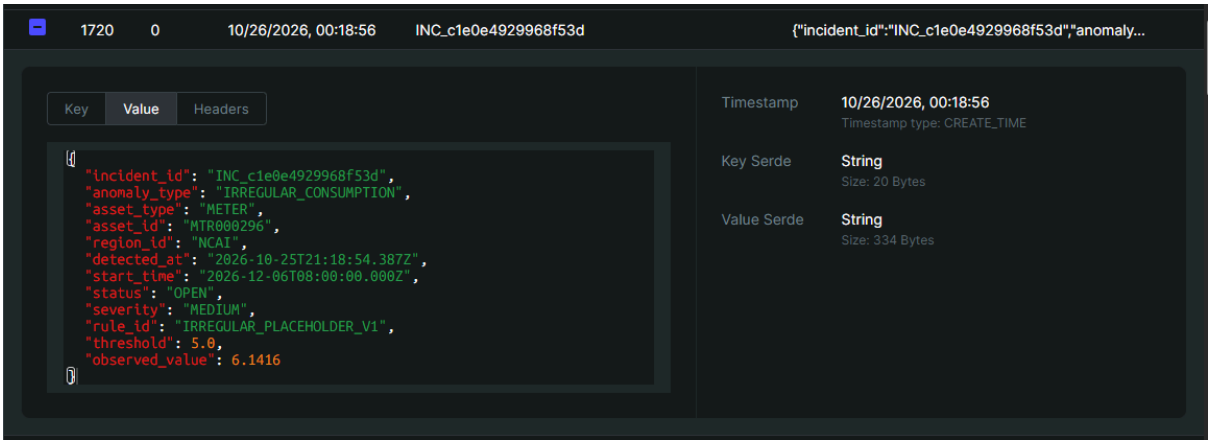
- TRANSFORMER_OVERLOAD (critical). Per transformer, per 15-minute window: estimated load = square root of (sum of kW squared + sum of kvar squared). Loading % = estimated load divided by rated capacity, times 100. Alert when loading exceeds `OVERLOAD_PCT` (default 100).
- VOLTAGE_DROP (high). Per meter, in a 45-minute window that slides every 15 minutes: alert when at least 3 readings are below `VOLTAGE_LOW_V` (default 210 V).
- IRREGULAR_CONSUMPTION (medium). Per reading: alert when `energy_kwh` exceeds 5.0, or when power is above 0.1 kW while energy is exactly 0. This rule is a placeholder. The intended rule compares each meter with its own baseline, which the batch job will build.

Each alert contains `incident_id` (a hash of type, asset and start time, so the same incident always gets the same id), `anomaly_type`, `asset_type`, `asset_id`, `region_id`, `detected_at`, `start_time`, `status`, `severity`, `rule_id`, `threshold` and `observed_value`. Overload and voltage alerts can be sent more than once while their window fills, so readers must upsert on `incident_id`.



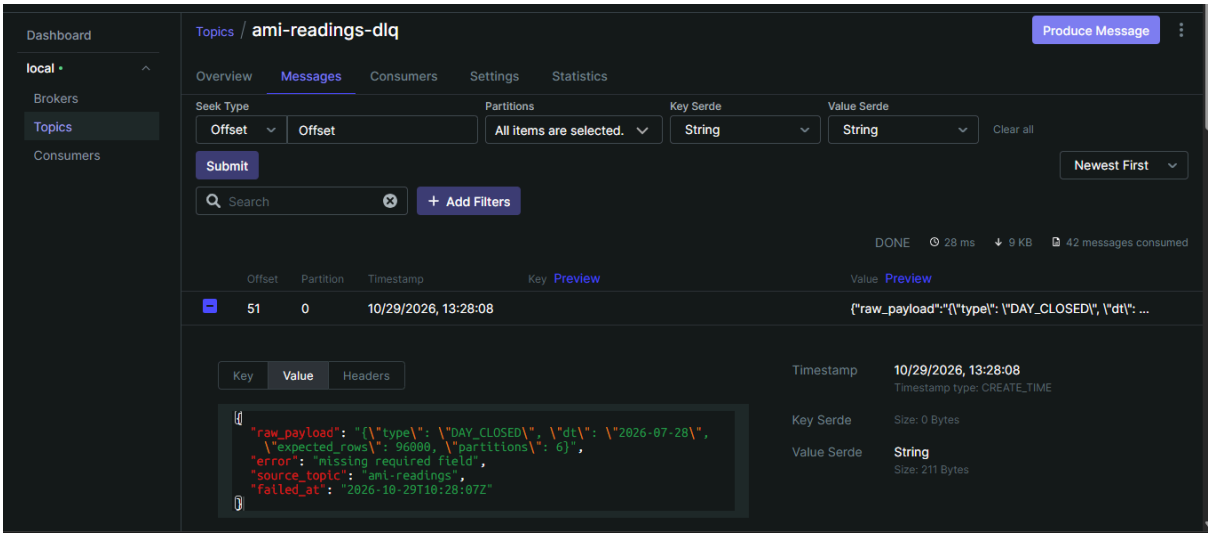

UI for Apache Kafka
83b5a60 v0.7.2
Dashboard
local
Brokers
Topics
Consumers
Topics / anomaly-alerts
Produce Message
Overview
Messages
Consumers
Settings
Statistics
Seek Type
Offset
Partitions
All items are selected.
Key Serde
String
Value Serde
String
Clear all
Submit
Newest First
Search
Add Filters
DONE
47 ms
35 KB
102 messages consumed
Offset
Partition
Timestamp
Key Preview
Value Preview
1698
2
10/26/2026, 00:19:00
INC_916abc0e25c6f355
{"incident_id":"INC_916abc0e25c6f355","anomaly...
Key
Value
Headers
{
"incident_id": "INC_916abc0e25c6f355",
"anomaly_type": "TRANSFORMER_OVERLOAD",
"asset_type": "TRANSFORMER",
"asset_id": "TX_NCAI_0003",
"region_id": "NCAI",
"detected_at": "2026-10-25T21:18:54.918Z",
Timestamp
10/26/2026, 00:19:00
Timestamp type: CREATE_TIME
Key Serde
String
Size: 20 Bytes
Value Serde
String
Size: 348 Bytes


1698
2
10/26/2026, 00:19:00
INC_916abc0e25c6f355
{"incident_id":"INC_916abc0e25c6f355","anomaly...
Key
Value
Headers
{
"incident_id": "INC_916abc0e25c6f355",
"anomaly_type": "TRANSFORMER_OVERLOAD",
"asset_type": "TRANSFORMER",
"asset_id": "TX_NCAI_0003",
"region_id": "NCAI",
"detected_at": "2026-10-25T21:18:54.918Z",
"start_time": "2026-12-06T08:00:00.000Z",
"status": "OPEN",
"severity": "CRITICAL",
"rule_id": "TX_OVERLOAD_V1",
"threshold": 100.0,
"observed_value": 115.06000118025726
}
Timestamp
10/26/2026, 00:19:00
Timestamp type: CREATE_TIME
Key Serde
String
Size: 20 Bytes
Value Serde
String
Size: 348 Bytes


1720
0
10/26/2026, 00:18:56
INC_c1e0e4929968f53d
{"incident_id":"INC_c1e0e4929968f53d","anomaly...
Key
Value
Headers
{
"incident_id": "INC_c1e0e4929968f53d",
"anomaly_type": "IRREGULAR_CONSUMPTION",
"asset_type": "METER",
"asset_id": "MTR000296",
"region_id": "NCAI",
"detected_at": "2026-10-25T21:18:54.387Z",
"start_time": "2026-12-06T08:00:00.000Z",
"status": "OPEN",
"severity": "MEDIUM",
"rule_id": "IRREGULAR_PLACEHOLDER_V1",
"threshold": 5.0,
"observed_value": 6.1416
}
Timestamp
10/26/2026, 00:18:56
Timestamp type: CREATE_TIME
Key Serde
String
Size: 20 Bytes
Value Serde
String
Size: 334 Bytes



ami-reading-dlq:



Testing

- **Rejection:** malformed JSON, a missing field, null voltage, zero voltage, negative energy and a bad timestamp were all sent to the DLQ with the correct reason. None of them reached Bronze.
- **Unusual valid data:** a 190 V reading was stored in Bronze and not rejected.
- **Duplicates:** the same reading sent twice appeared once in Bronze.
- **Late data:** a reading 15 minutes behind the newest event time was stored. A reading 90 minutes behind was dropped and does not appear in Bronze.
- **Irregular consumption:** a reading with 9.0 kWh produced an alert immediately.
- **Voltage drop:** three consecutive readings at 200 V produced one alert, starting at the first reading.
- **Transformer overload:** the alert fired, and the first value was wrong (see problem 4 below).

Batch Processing

1. Overview

The Batch Processing component is responsible for processing historical smart meter data from MinIO through Databricks, transferring the processed data to Snowflake, and preparing analytical datasets for Power BI dashboards.

The pipeline uses Databricks for data ingestion and Bronze-to-Silver transformations, while Snowflake stores the processed data and provides Gold analytical views. Apache Airflow is planned to orchestrate the end-of-day processing workflow for streaming data, applying the same processing logic and making the latest analytical results available to the dashboards.

2. Historical Data Ingestion: MinIO to Databricks

Historical Advanced Metering Infrastructure (AMI) data is stored in MinIO in Parquet format and organized into date-based partitions. The data is ingested into Databricks to support distributed processing and downstream transformations.

The ingestion stage includes:

- Reading historical smart meter readings from MinIO.
- Accessing the required reference datasets.
- Preserving relevant source columns, identifiers, and timestamps.
- Making the ingested data available for downstream transformations.
- Validating input availability before processing.

This stage connects the Bronze object storage layer with the Databricks processing environment.

3. Data Processing and Transformations in Databricks

Databricks processes the ingested data using a Bronze-to-Silver workflow. The objective is to transform the source data into clean, consistent, and validated datasets suitable for analytical consumption.

3.1 Bronze Layer

The Bronze layer contains the source data ingested from MinIO. It provides the initial processing layer while preserving the information required for traceability and validation.

The Bronze layer serves as the input for the subsequent cleansing and transformation steps.

3.2 Silver Layer

The Silver layer contains processed data derived from the Bronze layer.

The transformation workflow includes:

- Data cleansing and standardization.
- Schema and data type validation.
- Handling missing, invalid, or inconsistent records according to the project's data quality rules.
- Duplicate detection and deduplication.
- Applying data quality checks and quality flags.
- Validating smart meter identifiers and relevant reference relationships.
- Enriching readings using the required reference datasets.
- Preparing consistent datasets for downstream analytical processing.

These transformations produce the Silver datasets that are transferred to Snowflake.

4. Data Transfer: Databricks to Snowflake

After Silver processing, the required processed datasets are transferred from Databricks to Snowflake for analytical storage and querying.

The transfer stage includes:

- Selecting the required processed datasets.
- Loading the data into the corresponding Snowflake tables.
- Maintaining schema and data type consistency.
- Validating data availability and completeness.
- Preparing the warehouse data for the Gold analytical views.

This integration separates distributed data processing in Databricks from analytical data serving in Snowflake.

5. Gold Layer and Analytical Views in Snowflake

The Gold layer is implemented through analytical views in the `SMART_ENERGY_DB.MARTS` schema. These views organize the processed data into datasets that support energy consumption analysis, grid performance monitoring, voltage quality analysis, and data quality reporting.

The Gold views include:

5.1 `GOLD_DAILY_ENERGY_BY_REGION`

Provides daily energy consumption metrics grouped by region. It supports regional comparisons and analysis of energy consumption trends over time.

5.2 `GOLD_TRANSFORMER_LOAD_DAILY`

Provides daily transformer loading information to support monitoring of transformer utilization and identification of potential overload conditions.

5.3 GOLD_VOLTAGE_QUALITY_DAILY

Provides daily voltage quality indicators for analyzing voltage-related issues and monitoring power quality.

5.4 GOLD_METER_DAILY_CONSUMPTION

Provides daily consumption metrics at the smart meter level, supporting individual meter analysis and comparisons across dates.

5.5 GOLD_HOURLY_ENERGY_DEMAND

Provides hourly energy demand metrics to support analysis of demand patterns and peak-demand periods.

5.6 GOLD_TRANSFORMER_PERFORMANCE

Provides analytical information about transformer performance to support grid monitoring and performance comparisons.

Together, these views form the Gold analytical layer. They expose prepared datasets for reporting and reduce the need to repeatedly apply the same analytical logic to the underlying processed data.

6. Apache Airflow Orchestration for Daily Streaming Processing

Apache Airflow is planned to orchestrate the end-of-day workflow for streaming data stored in MinIO.

After the daily streaming data becomes available, Airflow will coordinate the processing pipeline so that the new data follows the same transformation and analytical workflow as the historical data.

Airflow acts as the orchestration layer. Databricks performs the data transformations, Snowflake stores the processed data and provides the Gold views, and Power BI presents the analytical results.

7. End-to-End Architecture

The overall data flow is:

Historical Data in MinIO —> Databricks —> Bronze-to-Silver Transformations —> Processed Silver Data —> Snowflake —> SMART_ENERGY_DB.MARTS —> Power BI Dashboards

Streaming Data in MinIO —> Apache Airflow (End-of-Day) —> Databricks —> Bronze-to-Silver Transformations —> Snowflake —> Updated Gold Views —> Power BI Data Refresh —> Updated Dashboards

9. Expected Outcome

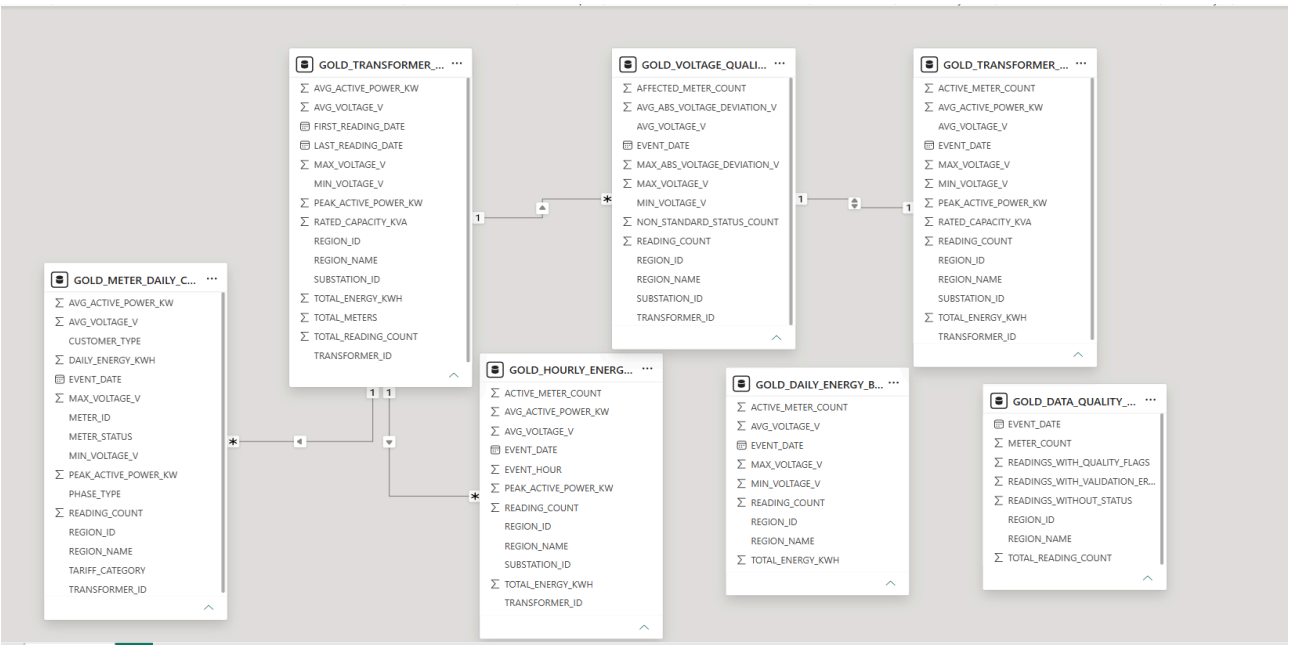
The pipeline provides a consistent approach for processing historical and streaming smart meter data. It connects MinIO object storage, Databricks transformations, Snowflake analytical views, and Power BI dashboards.

The end-of-day orchestration workflow is designed to process newly available streaming data using the same logic as historical data and make the resulting analytical outputs available through the existing Gold views and dashboards.

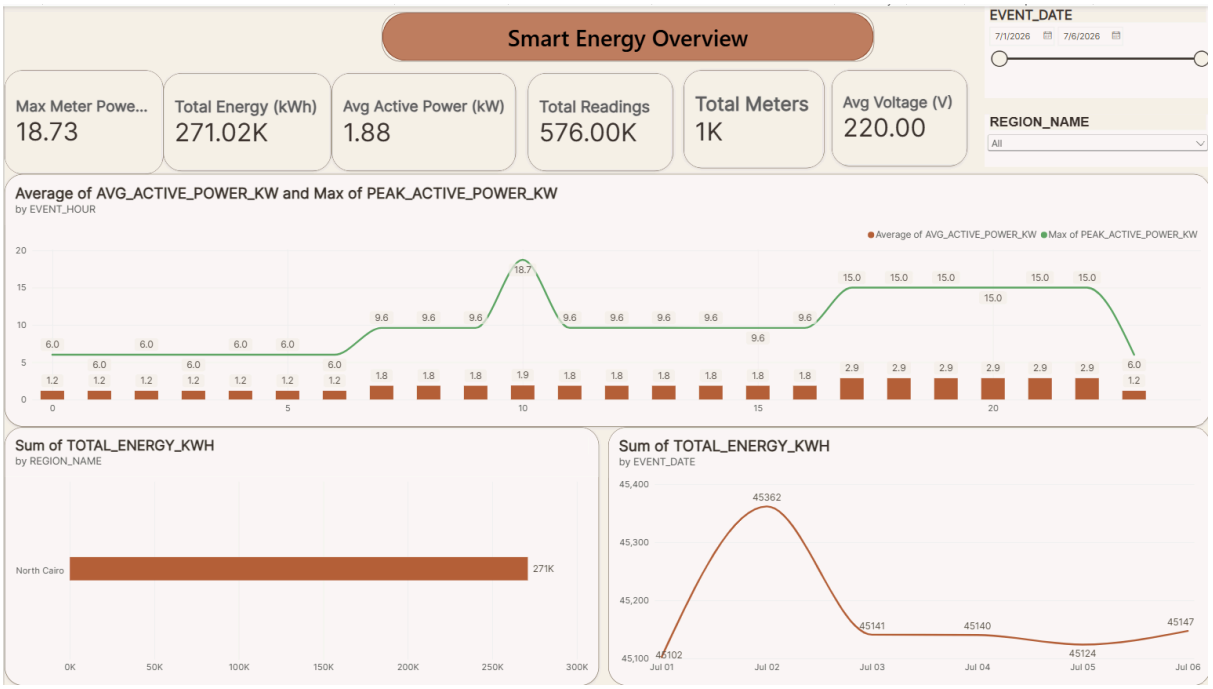
This architecture supports energy consumption analysis, grid performance monitoring, voltage quality analysis, and data quality reporting through a repeatable data processing workflow

Data Modeling & Power BI Integration

Power BI Data Model Architecture & Schema Relationships



- **Data Ingestion & Connectivity:**
  The data was ingested directly from **Snowflake Data Warehouse**, specifically targeting the **Gold Layer (`MARTS` Schema)** which contains cleaned, aggregated, and business-ready data views. The connection was established using Power BI's **Import Mode** to optimize query performance, enable offline data manipulation, and ensure fast visual rendering within the analytical dashboards.
- **Data Modeling & Relationship Architecture:**
  The data model is structured around key operational entities (e.g., daily energy consumption, hourly load demand, transformer statistics, and data quality metrics). Relationships between tables were defined using surrogate and primary keys—such as `TRANSFORMER_ID`, `REGION_ID`, and `EVENT_DATE`—enabling seamless cross-filtering and multi-dimensional analysis without data duplication.
- **Analytical Readiness:**
  Independent aggregated tables (such as regional daily summaries and quality logs) were preserved as standalone units within the schema. This structured data model serves as the foundational backbone for calculating key performance indicators (KPIs) and constructing the interactive decision-support dashboards.



Purpose. An executive summary of fleet-wide consumption, demand shape, and data health for the selected period and region. All values are simulated AMI readings at 15-minute intervals.

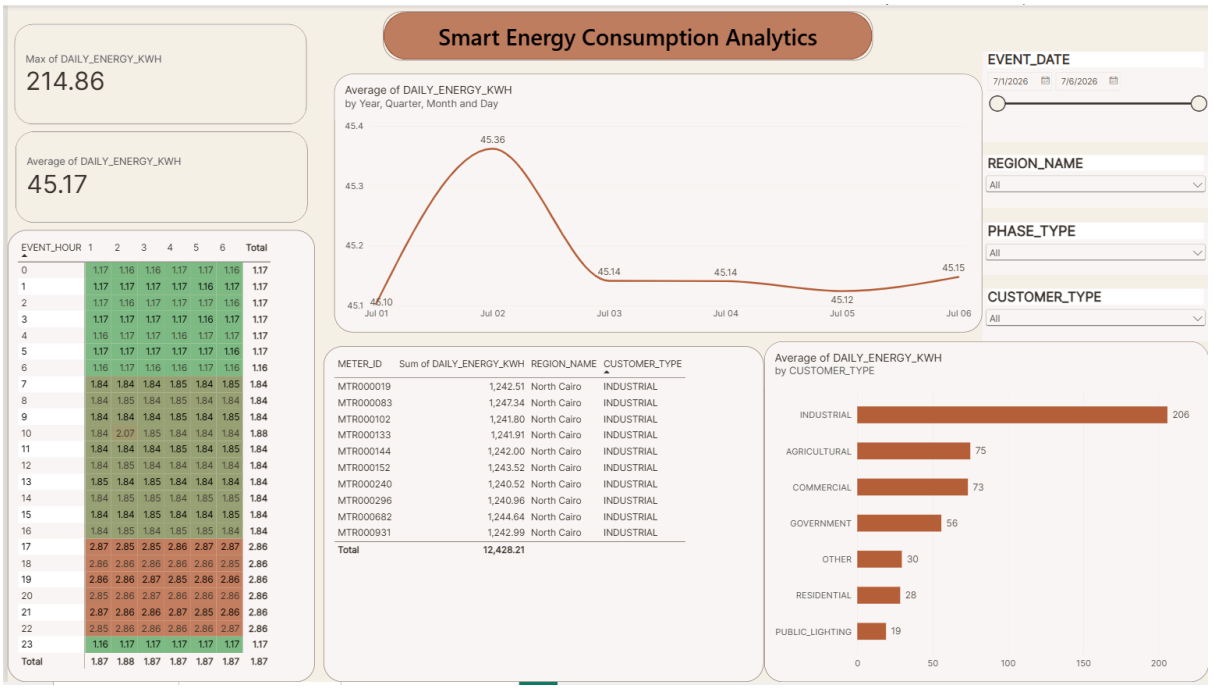
Filters. `EVENT_DATE` range and `REGION_NAME`, applied to all visuals on the page.

KPIs. Total Energy (kWh) is the sum of interval energy; Avg Active Power (kW) is the mean of `AVG_ACTIVE_POWER_KW`; Total Readings counts all readings (expected = meters × 96 × days); Total Meters is the distinct meter count; Avg Voltage (V) is the mean, not the sum, of `AVG_VOLTAGE_V`; Max Meter Power (kW) is the maximum single-meter reading.

Visuals. (1) Hourly demand profile: average and maximum active power by `EVENT_HOUR` (0–23). (2) Energy by region. (3) Daily energy trend by `EVENT_DATE`.

Sanity checks. 1,000 meters × 96 × 6 days = 576,000 readings; 1.88 kW × 24 h × 6 days × 1,000 meters ≈ 270.7 MWh, matching the 271.02K kWh total.

Limitations. Max Meter Power reflects a single meter, not aggregate peak demand. The sample covers one region and six days.



Purpose. Analyzes consumption behavior by time of day, customer segment, and individual meter, to establish segment-level baselines for anomaly detection. All values are simulated.

Filters. `EVENT_DATE`, `REGION_NAME`, `PHASE_TYPE`, `CUSTOMER_TYPE`.

KPIs. Max and Average of `DAILY_ENERGY_KWH` (grain: one meter per day, in kWh). Average = 45.17 kWh.

Visuals.

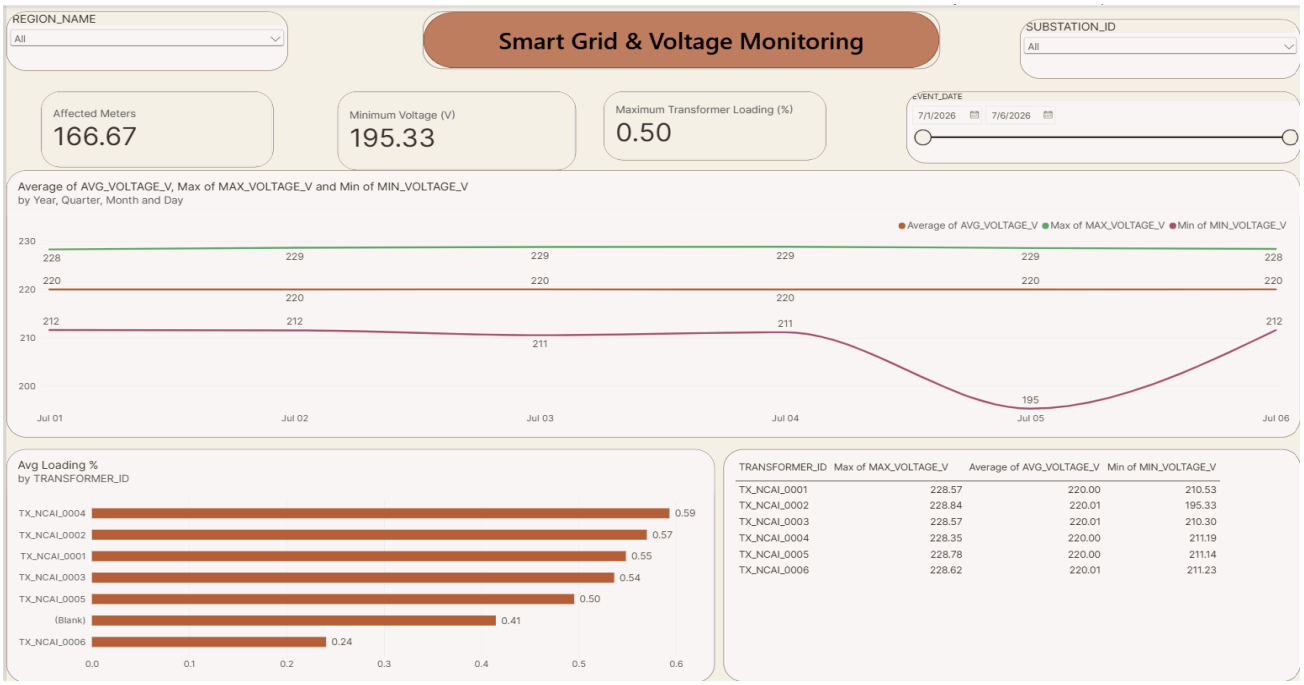
1. Daily trend of average per-meter energy (kWh) by `EVENT_DATE`.
2. Hour × day heatmap of average active power (kW) from the hourly demand table; color scale low to high.
3. Top 10 meters by total `DAILY_ENERGY_KWH` over the selected period.
4. Average `DAILY_ENERGY_KWH` by `CUSTOMER_TYPE`.

Key observations. Three-level load profile: night ~1.17 kW, daytime ~1.84 kW, evening peak (17:00–22:00) ~2.86 kW (≈2.4× night). Industrial meters average 206 kWh/day versus 28 kWh/day for residential (≈7×). A single deviating cell (hour 10, Jul 2: 2.07 kW vs ~1.84 kW) accounts for the +~230 kWh excess on that day.

Consistency checks. 45.17 kWh × 1,000 meters × 6 days ≈ 271,020 kWh, matching the Energy Overview total. The Top-10 total (12,428.21) equals the sum of its rows.

Limitations.

- The heatmap uses the hourly demand table, which has no `CUSTOMER_TYPE`/`PHASE_TYPE` columns, so those two slicers do not filter it.
- Column headers 1–6 are days of the month.
- The profile is step-shaped because the data is simulated; real AMI data would be smoother.



Purpose. Monitors voltage quality and transformer loading per transformer and per day, to surface voltage drops and capacity stress. All data is simulated.

Filters. `REGION_NAME`, `SUBSTATION_ID`, `EVENT_DATE`.

KPIs. Minimum Voltage (V) = minimum of `MIN_VOLTAGE_V`; Maximum Transformer Loading (%) = highest per-transformer Avg Loading; Meters per Transformer = average meters reporting per transformer.

Visuals.

1. Daily Voltage (avg/max/min of `AVG_VOLTAGE_V`, `MAX_VOLTAGE_V`, `MIN_VOLTAGE_V`) by `EVENT_DATE`.
2. Avg Loading % by `TRANSFORMER_ID`, defined as average active power (kW) divided by `RATED_CAPACITY_KVA`.
3. Transformer voltage table (Max, Avg, Min).

Key observations. Nominal voltage 220 V; normal daily minimum ≈ 210–211 V (≈4% below nominal); one deviation on Jul 5 (195.33 V, ≈11% below nominal) on `TX_NCAI_0002`. Average voltage stays ≈ 220 V, so averages alone mask voltage drops. Avg Loading ranges from 24% to 59%.

Limitations.

- Loading is average-based and approximates kVA by kW, so it cannot reveal short overload events (which push aggregate load to 105–120% of rated capacity for one or more 15-minute intervals).
- A peak-interval loading metric requires a Gold table with per-transformer aggregate load per interval.